Network Sniffing - Study Guide

Concepts, Techniques, Tools, Countermeasures & Lab Walkthroughs

Ethical Hacking / Network Security Study Notes
November 2026

1. Sniffing Concepts
2. Sniffing Technique: MAC Attacks
3. Sniffing Technique: DHCP Attacks
4. Sniffing Technique: ARP Poisoning
5. Sniffing Technique: Spoofing Attacks
6. Sniffing Technique: DNS Poisoning
7. Sniffing Tools
8. Countermeasures (General, Summary)
9. Sniffing Detection Techniques
10. Lab Walkthrough: Perform MAC Flooding Using `macof`
11. Lab Walkthrough: Perform a DHCP Starvation Attack
12. Lab Walkthrough: Perform ARP Poisoning
13. Lab Walkthrough: Perform a MITM Attack Using Cain & Abel
14. Lab Walkthrough: Spoof a MAC Address
15. Lab Walkthrough: Perform Password Sniffing

Quick Revision Summary

> ***Scope & Ethics Notice*** *This guide is for educational / certification-study purposes (e.g., CEH-style exam prep) and for authorized security testing only. Every tool and technique below must only be used in a lab you own or control, or on systems you have* ***explicit written permission*** *to test. Unauthorized interception of network traffic is illegal in most jurisdictions.*

1. Sniffing Concepts

1.1 What is Sniffing?

Sniffing is the process of monitoring and capturing data packets as they travel across a network using a software application or hardware device (a "sniffer"). Network administrators use it for troubleshooting and traffic analysis; attackers use it to capture sensitive data such as credentials, session tokens, and confidential files.

1.2 How Sniffing Works

A Network Interface Card (NIC) normally accepts only frames addressed to its own MAC address (and broadcast/multicast frames). A sniffer places the NIC into **promiscuous mode** (or **monitor mode** for wireless), causing it to capture *all* frames on the segment it can see, regardless of destination address.

1.3 Types of Sniffing

Type	Description
Passive Sniffing	Sniffing on a hub-based (shared-collision-domain) network. Every host sees every frame, so the attacker simply listens - no packets are injected, making it hard to detect.
Active Sniffing	Sniffing on a **switched** network. Because a switch forwards frames only to the intended port (per its CAM table), the attacker must actively inject traffic (e.g., flood the CAM table, poison ARP caches, spoof DHCP) to redirect traffic to themselves.

1.4 Why Switched Networks Change the Game

Switches build a **CAM (Content Addressable Memory) table** mapping MAC addresses to physical ports, so unicast frames are forwarded only to the correct port. This limits passive sniffing to broadcast/multicast traffic and traffic destined for the attacker's own port - hence the need for *active* techniques (MAC flooding, ARP poisoning, DHCP attacks, spoofing) covered in later sections.

1.5 Protocols Vulnerable to Sniffing

Sniffing is most dangerous against protocols that transmit data in **clear text**:

- **HTTP** – credentials and form data sent unencrypted
- **Telnet / Rlogin** – keystrokes, including passwords, sent in clear text
- **FTP** – usernames/passwords sent unencrypted
- **POP / IMAP / SMTP** – email credentials and content often in clear text
- **SNMP v1/v2** – community strings sent in clear text
- **NNTP** – passwords sent as clear text

1.6 Hardware Protocol Analyzers

A hardware protocol analyzer is a standalone device that plugs into a network to monitor traffic without altering it, used for in-depth performance and fault diagnosis. It captures and decodes data at the bit/byte level and is typically used by network engineers for troubleshooting, independent of a host OS.

1.7 SPAN Port (Switched Port Analyzer)

A SPAN (or “mirror”) port is a switch feature that copies traffic from one or more source ports (or VLANs) to a designated monitoring port, allowing an administrator (or attacker with access) to capture traffic legitimately without ARP poisoning - commonly used by IDS/IPS and packet analyzers such as Wireshark.

1.8 Wiretapping

- **Lawful Interception (LI)**: Legally sanctioned, officially approved network traffic interception for surveillance by government or law-enforcement agencies.
- **Active Wiretapping**: Alters or injects data into a communication (e.g., man-in-the-middle).
- **Passive Wiretapping**: Only monitors/records the traffic without modification.

2. Sniffing Technique: MAC Attacks

2.1 MAC Address Table (CAM Table)

A switch's CAM table maps **MAC address → switch port → VLAN**. It has a **fixed, finite size**. Normal operation: the switch learns a source MAC on a port, stores the mapping, and forwards future frames for that MAC only to that port.

2.2 MAC Flooding

MAC flooding exploits the CAM table's limited size. The attacker floods the switch with frames containing thousands of **forged (random) source MAC addresses**. The CAM table fills up and can no longer learn legitimate mappings.

Effect: Once the table is full, many switches **fail open** - they behave like a hub and broadcast all incoming frames to every port (this is sometimes called "fail-open mode"). This lets the attacker passively sniff traffic meant for other hosts on the same switch, even though the network is switched.

Common tool: `macof` (part of the `dsniff` suite) - generates thousands of frames per minute with random source MAC/IP pairs.

2.3 Switch Port Stealing

A technique where the attacker floods the network with forged gratuitous ARP packets using the **victim's MAC address as the source** and the **attacker's own MAC as destination**, racing the switch into re-mapping the victim's MAC to the attacker's port, causing the switch to send the victim's traffic to the attacker momentarily before the real host re-registers.

2.4 Mitigations for MAC Attacks

- **Port Security**: Limit the number of MAC addresses allowed to be learned per switch port (e.g., `switchport port-security maximum 2`).
- **Sticky MAC**: Statically bind the first-learned MAC address to a port.
- **Disable unused ports.**
- **802.1X port-based authentication.**
- Configure a violation action (shutdown/restrict/protect) when the MAC limit is exceeded.

3. Sniffing Technique: DHCP Attacks

3.1 DHCP Overview (DORA)

DHCP assigns IP configuration dynamically via a four-step handshake: **D**iscover → **O**ffer → **R**equest → **A**cknowledge (DORA). DHCP has no built-in authentication of clients or servers.

3.2 DHCP Starvation Attack

The attacker broadcasts a flood of `DHCPDISCOVER` messages using **spoofed, random source MAC addresses**, requesting and consuming every IP address in the DHCP server's scope.

Effect: The address pool is exhausted, so legitimate clients cannot obtain an IP lease - a Denial of Service. It is also commonly used as **setup** for a rogue DHCP server attack.

Common tools: `Yersinia`, `Gobbler`, `DHCPStarv`, `dhcpx`.

3.3 Rogue DHCP Server Attack

After starving the legitimate server's pool (or simply racing it), the attacker stands up a **rogue DHCP server** on the segment. Because DHCP clients accept the first offer received, the attacker's server can hand out IP leases where it sets itself as the **default gateway and/or DNS server**, routing all victim traffic through the attacker for sniffing/MITM.

3.4 Mitigations for DHCP Attacks

- **DHCP Snooping**: A switch feature that classifies ports as *trusted* (connected to legitimate DHCP servers) or *untrusted* (connected to hosts); DHCP server messages (OFFER/ACK) from untrusted ports are dropped.
- **Port Security** to limit MAC addresses per port (mitigates starvation).
- **Rate-limit** DHCP traffic per port.
- Build a **DHCP snooping binding table** (IP–MAC–port–lease) used to validate subsequent ARP (via Dynamic ARP Inspection) and IP Source Guard.

4. Sniffing Technique: ARP Poisoning

4.1 ARP Basics

Address Resolution Protocol (ARP) maps a known IP address to the corresponding MAC address on a LAN. ARP is **stateless**: a host will accept and cache ARP replies even if it never sent a corresponding request.

4.2 ARP Poisoning (ARP Spoofing)

The attacker sends forged (**gratuitous**) ARP replies to the victim(s), associating the **attacker's MAC address** with the **IP address of a legitimate host** (typically the default gateway). The victim updates its ARP cache with the false mapping.

Effect: Traffic the victim intends for the gateway (or another host) is instead sent to the attacker, who can then: 1. **Sniff** the traffic, and 2. **Forward** it on to the real destination (IP forwarding enabled) so the victim notices no interruption - a classic **Man-in-the-Middle (MITM)** setup.

4.3 Threats from ARP Poisoning

- Packet sniffing / credential theft
- Session hijacking
- VoIP call interception
- Data (manipulation) and MITM attacks
- Denial of Service (poisoning with a nonexistent MAC)

4.4 Common Tools

`Ettercap`, `Cain & Abel`, `arpspoof` (dsniff suite), `BetterCAP`, `Arpoison`.

4.5 Mitigations for ARP Poisoning

- **Dynamic ARP Inspection (DAI)**: Validates ARP packets against the DHCP snooping binding table; drops invalid ARP.
- **Static ARP entries** for critical hosts (gateway, servers) - not scalable but effective for small networks.
- Use **ARP spoofing detection tools** (e.g., XArp).
- Enable **port security** and **DHCP snooping** (DAI depends on it).
- Use encrypted protocols (HTTPS, SSH) so sniffed data is unusable even if intercepted.
- Physically secure switch ports / use 802.1X.

5. Sniffing Technique: Spoofing Attacks

5.1 MAC Spoofing / Duplicating

The attacker changes their NIC's MAC address to match that of a legitimate, authorized device, bypassing MAC-based access controls (e.g., MAC filtering on Wi-Fi) or impersonating the device to intercept its traffic.

How it's done: Sniff the network to identify a legitimate client's MAC, then change the attacker's NIC MAC to match it using OS utilities or tools such as `TMAC`, `Technitium MAC Address Changer`, or Linux's `macchanger` / `ip link`.

5.2 IP Address Spoofing

Crafting IP packets with a forged **source IP address**, so the destination believes the traffic originates from a trusted host. Used to bypass IP-based authentication, to perform DoS reflection attacks, and to hide the attacker's real identity.

Types: - **Non-blind spoofing** – attacker is on the same subnet, can see traffic/sequence numbers. - **Blind spoofing** – attacker is off-subnet and must predict sequence numbers. - **Man-in-the-middle spoofing** - **Denial-of-Service spoofing** (e.g., amplification/reflection using spoofed victim source IP)

5.3 DNS Spoofing

Covered in depth in Section 6 (DNS Poisoning) - providing false DNS responses so a victim resolves a domain to the attacker's IP.

5.4 Mitigations for Spoofing

- **Ingress/Egress filtering** (e.g., BCP 38) to drop packets with illegitimate source addresses at network edges.
- **Encryption and mutual authentication** (IPsec, TLS, SSH) instead of trust based on IP/MAC alone.
- **Port security / 802.1X** to prevent MAC spoofing on switch ports.
- Use **Reverse Path Forwarding (RPF)** checks on routers.

6. Sniffing Technique: DNS Poisoning

6.1 What is DNS Poisoning (DNS Cache Poisoning)?

DNS poisoning corrupts a DNS resolver's cache (or a victim's local host resolution) with a **false IP address** for a given domain, so that traffic intended for a legitimate site (e.g., a bank) is redirected to an attacker-controlled server - often a phishing clone used to harvest credentials, i.e., **Pharming**.

6.2 Types of DNS Poisoning

Technique	Description
Intranet DNS Spoofing	Performed on a switched LAN, typically combined with ARP poisoning so the attacker sees DNS queries and races a forged reply to the victim before the real DNS server responds.
Internet DNS Spoofing	Attacker compromises a victim's machine (e.g., malware) and changes its DNS server settings or hosts file to point to a rogue DNS server.
Proxy Server DNS Poisoning	Victim's browser proxy settings are changed (e.g., via malware) to route through a malicious proxy that returns forged DNS responses.
DNS Cache Poisoning	Forged data is injected into a legitimate DNS server's cache (e.g., via Kaminsky-style transaction-ID/UDP-source-port guessing), affecting *all* clients that query that resolver.

6.3 How an Attacker Races a DNS Response

Because classic DNS over UDP identifies a reply only by a 16-bit transaction ID and the query's source port, an attacker who can predict/guess these (or who is positioned on-path via ARP poisoning) can send a forged response before the authentic server's answer arrives, and the resolver will accept whichever valid-looking response arrives first.

6.4 Mitigations for DNS Poisoning

- **DNSSEC** (DNS Security Extensions) – cryptographically signs DNS records so forged responses can be detected.
- Restrict **recursive DNS queries** to trusted clients; disable open recursion.
- Use **randomized transaction IDs and source ports** for DNS queries.
- Keep DNS server software patched; run DNS on isolated, hardened hosts.
- Configure resolvers/clients to ignore additional/unsolicited DNS records outside the requested zone.
- Use HTTPS with certificate validation so even a poisoned DNS redirect is caught by certificate mismatch.

7. Sniffing Tools

7.1 Categories

- **Packet analyzers / protocol analyzers**: Wireshark, tcpdump, Microsoft Network Monitor
- **ARP/MITM-focused suites**: Ettercap, Cain & Abel, BetterCAP
- **CLI toolkits**: dsniff suite (`macof`, `arpspoof`, `dsniff`, `mailsnarf`, `urlsnarf`, `webspy`)
- **Wireless sniffers**: Kismet, Aircrack-ng suite (`airodump-ng`)
- **Hardware taps**: inline network taps, SPAN-port based analyzers

7.2 Notable Tools Summary

Tool	Primary Use
Wireshark	The de-facto standard GUI packet capture/analysis tool; deep protocol dissection, filters (BPF), follow-TCP-stream.
tcpdump	Lightweight CLI packet capture tool, widely used on Linux/Unix for scripting and remote capture.
Ettercap	Full-featured MITM suite: ARP poisoning, DNS spoofing, SSL stripping, plugins, filters.
Cain & Abel	Windows password-recovery/sniffing suite with built-in ARP poisoning (“APR” - ARP Poison Routing) and protocol decoders.
dsniff / macof / arpspoof	Classic *nix auditing toolkit - MAC flooding, ARP spoofing, and password sniffing utilities.
BetterCAP	Modern, modular MITM framework (ARP/DNS spoofing, sniffing, credential harvesting, Wi-Fi attacks).
Kismet / Aircrack-ng	Wireless network discovery, packet capture, and 802.11 attack toolset.
SmartSniff / OmniPeek / CommView	Commercial/GUI sniffers with analysis and reporting features.

8. Countermeasures (General, Summary)

1. **Restrict physical access** to network closets, switches, and cabling.
2. **Use switches with port security** and disable unused switch ports.
3. **Implement DHCP Snooping + Dynamic ARP Inspection + IP Source Guard** together (they build on the same binding table).
4. **Use static ARP entries** for critical infrastructure where feasible.
5. **Encrypt everything in transit**: HTTPS/TLS, SSH instead of Telnet, SFTP/FTPS instead of FTP, IMAPS/SMTPS/POP3S.
6. **Use VPNs** for remote or untrusted-network communication.
7. **Deploy 802.1X** network access control so only authenticated devices can send traffic.
8. **Segment networks with VLANs** to shrink broadcast/collision domains.
9. **Enable DNSSEC** and restrict open DNS recursion.
10. **Monitor for promiscuous-mode NICs** and unusual ARP activity (IDS/IPS, `arpwatch`).
11. **Use HSTS and certificate pinning** for web applications to resist SSL-stripping MITM.
12. **Regularly audit CAM/ARP tables** and switch logs for anomalies.

9. Sniffing Detection Techniques

9.1 Detecting Promiscuous Mode

A NIC in promiscuous mode accepts frames not addressed to it, which can be detected with techniques such as:

- **Ping Method**: Send an ICMP echo request to the *suspect host's IP* but with an **incorrect MAC address**. A normal NIC drops the frame at the hardware level (wrong MAC) and never replies. A NIC in promiscuous mode may pass the frame up the stack, and if the host's software isn't filtering it either, it can reply - indicating promiscuous mode.
- **ARP Method**: Send a non-broadcast ARP request to the suspect machine (unicast, not broadcast) with the correct IP but a MAC not matching ARP cache rules. A host in promiscuous mode that is sniffing may still respond because it processes all frames.
- **DNS Method**: Many sniffers perform **reverse DNS lookups** on captured IP addresses. Generate traffic to a decoy IP and watch for an unexpected reverse-DNS (PTR) query appearing on the network - evidence a sniffer is resolving captured traffic.
- **Latency/Load Method**: Flood the network segment with a large volume of traffic; a host actively sniffing and processing every frame typically shows a measurable increase in response latency compared to baseline, since its CPU/stack is processing far more frames than normal.

9.2 Network-Based Detection

- **arpwatch / arp-scan**: Monitor and alert on unexpected IP–MAC pair changes.
- **IDS/IPS signatures** for known ARP-spoofing/MITM tool traffic patterns.
- **Port security violation logs** on switches (unexpected MAC counts per port).
- **NIC configuration audit tools** (e.g., `ifconfig` / `ip link` flag checks, `PromqryUI` on Windows) to directly query interfaces for promiscuous-mode flags across the network.

10. Lab Walkthrough: Perform MAC Flooding Using `macof`

> ***Lab-only***: *run exclusively inside an isolated virtual lab (e.g., an isolated vSwitch connecting attacker and victim VMs) that you own.*

10.1 Objective

Observe how flooding a switch's CAM table with forged MAC addresses can force it into fail-open (hub-like) behavior.

10.2 Prerequisites

- Linux attack machine with the `dsniff` package installed (`sudo apt install dsniff`).
- A lab switch (physical or virtual) connecting the attacker, a victim host, and a monitoring host.

10.3 Conceptual Steps

1. **Baseline capture**: On the attacker machine, start Wireshark/tcpdump on the lab interface and confirm that, in normal switched operation, you only see broadcast traffic and your own unicast traffic (not the victim's traffic to other hosts).
2. **Run macof** to generate a flood of frames with randomized source/destination MAC and IP pairs:

   ```
   macof -i eth0
   ```

   Optional flags: `-n <count>` limits the number of packets; `-s` / `-d` set specific source/destination IP ranges.
3. **Observe the switch**: Many switch CAM tables (small/unmanaged switches especially) fill within seconds, exhausting table capacity.
4. **Re-check captured traffic**: If the switch fails open, your capture on the attacker NIC will suddenly start showing frames destined for other hosts' MAC addresses - traffic it would not normally forward to your port.
5. **Stop the attack** (Ctrl+C) and allow the CAM table to age out / repopulate normally.

10.4 Expected Result

On a vulnerable switch, you see unicast traffic between other lab hosts arriving on your sniffing interface, demonstrating the fail-open weakness. On a switch with **port security** configured (e.g., `switchport port-security maximum 5`, violation action `shutdown`), the attacking port will instead be **error-disabled**, and no flood occurs.

10.5 Takeaway for Defense

This lab is the practical justification for Section 2.4's countermeasures - port security with a MAC-address cap is the direct, effective control against this attack.

11. Lab Walkthrough: Perform a DHCP Starvation Attack

> ***Lab-only**, isolated network with your own DHCP server (e.g., a test VM running `isc-dhcp-server` or a lab router).*

11.1 Objective

Exhaust a test DHCP server's address pool and observe the resulting denial of service, then test DHCP-snooping mitigation.

11.2 Conceptual Steps (using Yersinia, a classic CEH-lab tool)

1. Set up a small lab DHCP scope (e.g., 10 addresses) on a test server/router.
2. On the attacker Linux host, launch Yersinia's DHCP module:

   ```
   sudo yersinia dhcp -i eth0 -attack 1
   ```

   (In the Yersinia GUI: `Launch Attack → DHCP → sending DISCOVER packet` performs the equivalent starvation flood.)
3. Yersinia generates a stream of `DHCPDISCOVER` packets, each using a freshly randomized source MAC address, requesting a lease each time.
4. **Monitor the DHCP server logs**: watch the scope's "addresses leased/available" counter drop to zero.
5. **Test impact**: attempt to connect a new legitimate client to the segment and confirm it fails to obtain an IP (DHCP timeout).
6. **(Optional follow-on)** Stand up a rogue DHCP server (e.g., `dnsmasq` configured with attacker-controlled gateway/DNS options) on the now-starved segment to simulate a full rogue-server/MITM chain.

11.3 Expected Result

The legitimate DHCP server's pool reaches 0 available leases; new clients fail to obtain addresses, confirming the DoS.

11.4 Mitigation Test

Re-run the attack after enabling **DHCP snooping** and **port security** (MAC address rate-limiting) on the lab switch port connected to the attacker; the flood should now be rate-limited or the port disabled, and the legitimate pool should remain available.

12. Lab Walkthrough: Perform ARP Poisoning

> ***Lab-only**, isolated network with an attacker VM, a victim VM, and a gateway/router VM.*

12.1 Objective

Redirect a victim's gateway-bound traffic through the attacker machine to demonstrate a classic on-path (MITM) position.

12.2 Using `arpspoof` (dsniff suite)

1. Enable IP forwarding on the attacker so the victim's connectivity is preserved (otherwise the victim loses network access, which is noisy/detectable):

   ```
   sudo sysctl -w net.ipv4.ip_forward=1
   ```

2. Poison the victim, telling it the attacker's MAC is the gateway's:

   ```
   sudo arpspoof -i eth0 -t <victim_IP> <gateway_IP>
   ```

3. (Optional, for full bidirectional MITM) Poison the gateway as well, telling it the attacker's MAC is the victim's:

   ```
   sudo arpspoof -i eth0 -t <gateway_IP> <victim_IP>
   ```

4. **Verify on the victim**: check its ARP cache (`arp -a` on Windows or `ip neigh` on Linux) - the gateway's IP should now map to the attacker's MAC.

5. **Capture traffic** on the attacker with Wireshark/tcpdump while the victim browses, confirming visibility into the victim's packets before they are forwarded to the real gateway.

6. **Clean up**: stop `arpspoof` (Ctrl+C) and disable IP forwarding; allow ARP caches to age out and re-correct.

12.3 Using Ettercap (GUI alternative)

```
sudo ettercap -G
```

In the GUI: `Sniff → Unified sniffing → select interface`, then `Hosts → Scan for hosts`, add the victim to **Target 1** and the gateway to **Target 2**, then `Mitm → ARP poisoning → Sniff remote connections`, and finally `Start → Start sniffing`.

12.4 Expected Result

Victim traffic destined for the gateway is visible on the attacker's capture, and (with forwarding enabled) the victim experiences no loss of connectivity - demonstrating a transparent MITM position.

12.5 Mitigation Test

Configure **Dynamic ARP Inspection** on the lab switch (backed by DHCP snooping) and repeat the attack; forged ARP packets from the attacker's untrusted port should now be dropped, and the victim's ARP cache should remain correct.

13. Lab Walkthrough: Perform a MITM Attack Using Cain & Abel

> ***Lab-only**, Windows attacker VM on an isolated network. Cain & Abel is an older, Windows-only tool often used in classroom labs for conceptual illustration; it is largely unmaintained and frequently flagged by antivirus, so run it only in a disposable/offline lab VM.*

13.1 Objective

Use Cain & Abel's built-in **APR (ARP Poison Routing)** feature to intercept traffic between two lab hosts and inspect it with its protocol sniffers/decoders.

13.2 Conceptual Steps

1. Install Cain & Abel on the Windows attacker VM; launch it with administrator privileges.
2. Select the correct lab NIC under **Configure** and start the **Sniffer** tab (this places the NIC into promiscuous mode and begins host discovery).
3. Click the **+** icon to scan the subnet for live hosts (MAC/IP pairs populate the host list).
4. Switch to the **APR** tab, click **+**, and in the dialog select the **victim host** in the left column and the **gateway** in the right column (or vice-versa) to define the poisoning pair.
5. Click the **Start/Stop APR** (radiation) icon to begin ARP poisoning both directions.
6. Open the **Passwords** tab to view Cain's built-in protocol parsers automatically decoding any clear-text credentials (FTP, HTTP Basic Auth, Telnet, POP3, etc.) traversing the poisoned path.
7. Use the **Sniffer → RDP/SSH-1/HTTPS** sub-tabs to observe (where unencrypted or downgradable) additional session data, and the built-in **packet capture** to export a PCAP for later analysis in Wireshark.
8. When finished, click **Stop APR** to restore normal ARP mappings and stop the sniffer.

13.3 Expected Result

Clear-text credentials or session data generated by lab victim activity (e.g., a test FTP or HTTP login) appear decoded in Cain & Abel's Passwords tab, demonstrating the end-to-end impact of ARP poisoning + sniffing + protocol parsing.

13.4 Why This Fails Against Modern Defenses

- TLS/HTTPS-protected traffic shows up only as encrypted bytes - credentials are not recoverable.
- DAI/DHCP snooping on managed switches block the underlying ARP poisoning.
- Host-based ARP-spoof detection (e.g., XArp) alerts the victim to the changed gateway MAC.

14. Lab Walkthrough: Spoof a MAC Address

> *Changing your own device's MAC address for lab/testing purposes is generally legal on hardware you own; using it to **impersonate another device on a network you don't control** is not authorized.*

14.1 On Linux

```
# View current MAC
ip link show eth0

# Bring the interface down before changing
sudo ip link set dev eth0 down

# Set a new MAC address
sudo ip link set dev eth0 address 00:11:22:33:44:55

# Bring it back up and verify
sudo ip link set dev eth0 up
ip link show eth0
```

Alternative using `macchanger`:

```
sudo apt install macchanger
sudo macchanger -r eth0      # random MAC
sudo macchanger -m 00:11:22:33:44:55 eth0   # specific MAC
```

14.2 On Windows

1. Open **Device Manager → Network adapters**, right-click the adapter → **Properties**.
2. Go to the **Advanced** tab, select **Network Address** (or **Locally Administered Address**) from the property list.
3. Enter the desired 12-hex-digit MAC value and click **OK**.
4. Disable and re-enable the adapter (or reboot) for the change to take effect. (GUI tools such as **Technitium MAC Address Changer** or **TMAC** automate this.)

14.3 On macOS

```
sudo ifconfig en0 ether 00:11:22:33:44:55
```

14.4 Verifying the Change

```
ifconfig   # or ip link / ipconfig /all
```

Confirm the adapter now reports the new MAC, and that it reverts after an interface reset unless made persistent via a startup script (the hardware-burned MAC is unaffected - only the OS-presented value changes).

15. Lab Walkthrough: Perform Password Sniffing

> ***Lab-only.*** *Use a disposable test account and a clear-text protocol you control (e.g., a lab FTP or HTTP basic-auth login) - never attempt this against real credentials or production systems.*

15.1 Objective

Capture clear-text authentication traffic between a lab client and lab server to demonstrate why unencrypted protocols are unsafe on a sniffable network.

15.2 Setup

1. Stand up a lab service that authenticates in clear text, e.g., a test FTP server (`vsftpd`) or a simple HTTP Basic-Auth page.
2. Ensure the attacker machine is positioned to see the traffic - either on a hub/mirrored segment, or via the ARP-poisoning MITM position established in Section 12.

15.3 Capturing with Wireshark

1. Start Wireshark on the attacker interface: `Capture → Start`.
2. From the lab client, log in to the test FTP/HTTP service with a disposable test username/password.
3. Stop the capture and apply a display filter, e.g.:

   ```
   ftp
   ```

   or for HTTP Basic Auth:

   ```
   http.authorization
   ```

4. Right-click the relevant packet → **Follow → TCP Stream** to view the session in clear text, including the `USER` / `PASS` commands (FTP) or the Base64-encoded `Authorization` header (HTTP - easily decoded, as Base64 is encoding, not encryption).

15.4 Capturing with `dsniff`

The dsniff suite includes purpose-built clear-text credential sniffers:

```
sudo dsniff -i eth0          # generic credential sniffer (FTP, Telnet, HTTP, POP, IMAP, etc.)
sudo mailsnarf -i eth0       # reconstructs sniffed email
sudo urlsnarf -i eth0        # logs sniffed HTTP requests/URLs
```

15.5 Expected Result

The lab username and password appear in clear text (or trivially decodable Base64) in the capture - demonstrating concretely why clear-text protocols must be replaced with encrypted equivalents (FTPS/SFTP instead of FTP, HTTPS instead of HTTP, etc.).

15.6 Mitigation Test

Repeat the exercise against an **HTTPS** or **SFTP** test service instead; confirm that Wireshark now shows only encrypted TLS/SSH records with no recoverable credentials, validating Section 8's "encrypt everything in transit" countermeasure.

Quick Revision Summary

#	Topic	One-line takeaway
1	Sniffing Concepts	Promiscuous mode + shared medium = visibility; switches limit this, driving “active” attacks.
2	MAC Attacks	Flood the CAM table (`macof`) to force fail-open/hub behavior; mitigate with port security.
3	DHCP Attacks	Starve the DHCP pool, then run a rogue DHCP server to control gateway/DNS; mitigate with DHCP snooping.
4	ARP Poisoning	Forge ARP replies to redirect traffic through the attacker; mitigate with DAI + static ARP.
5	Spoofing Attacks	Forge MAC/IP source identity to bypass filters or impersonate trusted hosts; mitigate with filtering + encryption.
6	DNS Poisoning	Inject false DNS answers to redirect victims (pharming); mitigate with DNSSEC.
7	Sniffing Tools	Wireshark/tcpdump (capture/analyze), Ettercap/Cain & Abel/BetterCAP (MITM suites), dsniff (CLI toolkit).
8	Countermeasures	Physical security + switch hardening + encryption + 802.1X + monitoring.
9	Detection	Ping/ARP/DNS/latency methods detect promiscuous NICs; arpwatch/IDS detect ARP anomalies.
10–15	Labs	Hands-on reproduction of MAC flooding, DHCP starvation, ARP poisoning, Cain & Abel MITM, MAC spoofing, and password sniffing - always in an isolated, authorized lab.